

Redes de Comunicaciones y Cómputo Distribuido

2do Cuatrimestre 2026

Taller - Spanning Tree Protocol (STP)

1 Enunciado del taller

En este laboratorio se propone estudiar experimentalmente el siguiente problema:

¿Cómo coordinan los switches Ethernet la construcción de una topología libre de bucles y cómo reacciona la red ante fallas de enlace cuando se utiliza Spanning Tree Protocol?

Para comprender lo anterior, se deberá:

- Implementar una topología de red que contiene ciclos utilizando Containernet y Open vSwitch.
- Analizar el proceso de elección del root bridge.
- Analizar los roles y estados de los puertos en STP.
- Analizar cómo afecta STP al forwarding de tramas y cómo condiciona el aprendizaje de direcciones MAC.
- Inducir fallas en el enlace y medir los tiempos de convergencia de la red. Analizar cómo influyen los timers del protocolo STP.

2 Propósito general del taller

El objetivo del laboratorio es comprender experimentalmente el funcionamiento del algoritmo de Spanning Tree Protocol (STP) y analizar cómo este mecanismo permite mantener una topología de Capa 2 libre de bucles mientras conserva redundancia en la red.

3 Requisitos del sistema

El taller se ejecuta completamente dentro de un contenedor Docker, pero requiere:

- Linux (Ubuntu 22.04/24.04)
- Docker
- Permisos para ejecutar contenedores privilegiados

No es necesario instalar Mininet ni Open vSwitch.

4 Descarga de la imagen del taller

Para descargar la imagen preconstruida del taller, ejecute el siguiente comando:

```
docker pull jcbasso95/rccd-containernet
```

Esta imagen contiene:

- Containernet

- Mininet
- Open vSwitch
- Python 3
- Librerías necesarias para automatización y análisis

5 Ejecución del contenedor

Desde el directorio raíz del taller (`Taller_STP_Containernet/`), ejecute el contenedor con privilegios:

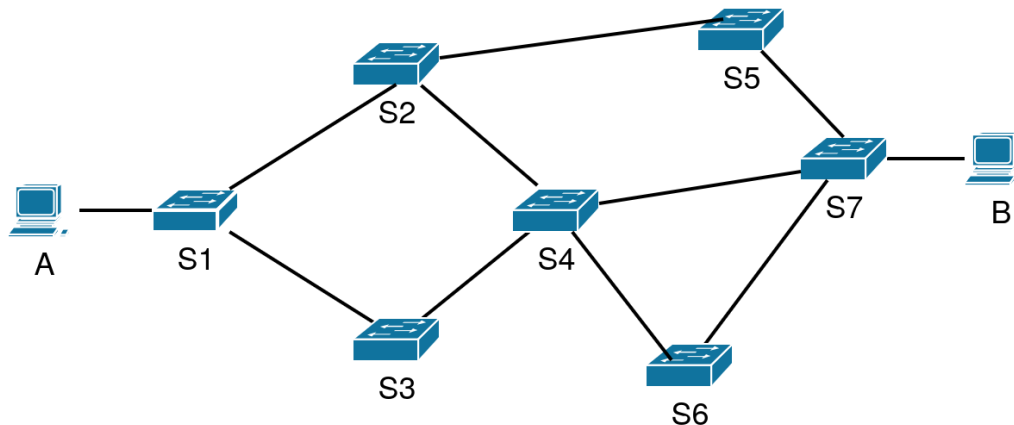
```
docker run --rm -it \  
  --name taller-stp \  
  --privileged \  
  -v /var/run/docker.sock:/var/run/docker.sock \  
  -v /lib/modules:/lib/modules \  
  -v $(pwd):/workshop \  
  jcbasso95/rccd-containernet
```

Dentro del contenedor, el directorio del taller estará disponible en:

```
/workshop
```

6 Construcción de la topología

La figura a continuación muestra la topología a usar en el taller.



Nota: en este taller, la numeración de los switches (S1 - S7) se usa solamente para referenciar un switch específico, no como el ID transmitido en un BPDU de STP. Para elegir entre varios switches cuando su distancia al root es la misma, esta implementación de STP usa el valor de `stp-priority` configurado en los switches, y en caso de que sean iguales desempata usando la dirección MAC de cada switch.

Una vez dentro del contenedor ejecute:

```
cd /workshop
python3 topo_sn_stp.py
```

Este script crea:

- 2 hosts (A y B)
- 7 switches Open vSwitch (s1 a s7)

- Múltiples enlaces redundantes formando ciclos de Capa 2
- Una topología diseñada para que STP sea obligatorio

Al finalizar, se ingresa automáticamente a la CLI de Containernet.

6.1 Objetivo de este paso

Construir una red con loops reales de Capa 2, condición necesaria para estudiar STP.

7 Verificación inicial de la topología

Una vez en la CLI de Containernet puede ejecutar:

```
nodes
net
```

Esto permite verificar:

- Qué nodos existen.
- Cómo están conectados.
- Qué interfaces tiene cada switch.

8 Activación de STP clásico (IEEE 802.1D)

El script `topo_sn_stp.py` habilita STP 802.1D y desactiva RSTP automáticamente en todos los switches al iniciar la topología.

Para asegurar que STP haya convergido antes de realizar pruebas, espere unos 45 segundos o ejecute en la CLI de Containernet:

```
sh sleep 45
```

En caso de querer forzar o verificar manualmente la configuración de STP y desactivación de RSTP en todos los bridges:

Desde una terminal bash (dentro del contenedor):

```
for s in $(ovs-vsctl list-br); do
  ovs-vsctl set bridge $s rstp_enable=false
  ovs-vsctl set bridge $s stp_enable=true
done
```

O desde la CLI de Containernet (`containernet>`):

```
sh for s in $(ovs-vsctl list-br); do ovs-vsctl set bridge $s rstp_enable=false; done
sh for s in $(ovs-vsctl list-br); do ovs-vsctl set bridge $s stp_enable=true; done
```

8.1 Objetivo de este paso

Forzar el uso de STP clásico, con timers largos y convergencia observable.

9 Inspección del estado STP

Para observar el árbol construido:

```
sh ovs-appctl stp/show
```

O para un switch específico (ejemplo, el central):

```
sh ovs-appctl stp/show s4
```

Analizar:

- Cuál es el Root Bridge.
- Cuál es el root-port de cada switch.
- Qué puertos están en forwarding.
- Qué puertos están en blocking.

9.1 Objetivo de este paso

Entender la topología lógica STP, que no coincide necesariamente con la topología física.

10 Prueba de conectividad inicial

El script de topología ya configura automáticamente los bridges en modo standalone con la regla de forwarding **NORMAL**.

Si en algún momento necesita restablecer este modo manualmente:

Desde una terminal bash (dentro del contenedor):

```
for s in $(ovs-vsctl list-br); do
  ovs-vsctl set bridge $s fail_mode=standalone
  ovs-ofctl add-flow $s "priority=0,actions=NORMAL"
done
```

O si se encuentra directamente en la CLI de Containernet (**containernet>**):

```
sh for s in $(ovs-vsctl list-br); do ovs-vsctl set bridge $s fail_mode=standalone; done
sh for s in $(ovs-vsctl list-br); do ovs-ofctl add-flow $s "priority=0,actions=NORMAL"; done
```

Desde la CLI de Containernet verifique conectividad ejecutando:

```
A ping -c 3 10.0.0.11
```

Si STP convergió correctamente, debería haber conectividad entre A y B.

11 Snapshot del estado STP

Para ejecutar comandos de apoyo o inspección sin interrumpir la CLI de Containernet, abra una segunda terminal en el host e ingrese al contenedor:

```
docker exec -it taller-stp bash
cd /workshop
```

Desde esta segunda terminal, ejecute:

```
python3 stp_snapshot.py --bridges s4
```

Este script guarda el estado STP del switch indicado en **results/**.

11.1 Objetivo de este paso

Tener una fotografía del árbol STP antes de realizar cambios o fallas.

12 Forzar el Root Bridge

Para forzar un switch como Root Bridge (ejemplo, s4) ejecute en la segunda terminal dentro del contenedor:

```
python3 set_root.py s4 4096
```

Volver a inspeccionar STP:

```
python3 stp_snapshot.py --bridges s4
```

o directamente desde la CLI de Containernet:

```
sh ovs-appctl stp/show s4
```

12.1 Objetivo de este paso

Observar cómo cambia el árbol STP al modificar prioridades de bridge.

13 Modificación de timers STP

Modificar los timers del switch central (ejecutar en la segunda terminal dentro del contenedor):

```
python3 set_stp_timers.py s4 1 10 6
```

Parámetros:

- hello-time = 1 s
- max-age = 10 s
- forward-delay = 6 s

Luego de unos segundos, inspeccionar la configuración de un switch no central con:

```
sh ovs-appctl stp/show s5
```

Observar que la configuración del bridge actualizó sus timers a los valores configurados en el switch central, de forma que todos los switches tengan una configuración consistente una vez que convergen.

13.1 Objetivo de este paso

Analizar cómo los timers afectan:

- Estabilidad.
- Velocidad de convergencia.
- Impacto ante fallas.

14 Inyección de fallas de enlace

Antes de inyectar las fallas, inicie un monitoreo continuo de conectividad.

```
A sh ping_monitor.sh
```

O alternativamente mediante el comando en una sola línea:

```
A sh -c 'while true; do ts=$(date -Iseconds); if ping -c 1 -W 1 10.0.0.11 >/dev/null 2>&1; then echo "$ts OK"; else echo "$ts FAIL"; fi; sleep 1; done | tee ping.log'
```

Este comando:

- Genera timestamps ISO.
- Escribe en el archivo `ping.log`.
- Permite medir convergencia STP.

Nota: Este proceso quedará ejecutándose en primer plano en la CLI de Containernet. Podrá detenerlo con `Ctrl+C` una vez finalizado el experimento.

En la segunda terminal dentro del contenedor ejecute:

```
python3 link_failure.py s4-eth4 15
```

Parámetro:

- `down_time = 15 s`

Este script permite generar una falla en el enlace y espera para que STP reaccione.

Luego de ejecutar el script anterior, debería observar en `ping.log` algo como:

```
2026-02-09T14:49:08+00:00 OK
2026-02-09T14:49:09+00:00 OK
2026-02-09T14:49:10+00:00 OK
2026-02-09T14:49:11+00:00 OK
2026-02-09T14:49:12+00:00 OK
2026-02-09T14:49:13+00:00 OK
2026-02-09T14:49:14+00:00 FAIL
2026-02-09T14:49:16+00:00 FAIL
2026-02-09T14:49:18+00:00 FAIL
2026-02-09T14:49:20+00:00 FAIL
2026-02-09T14:49:22+00:00 FAIL
2026-02-09T14:49:24+00:00 FAIL
2026-02-09T14:49:26+00:00 OK
2026-02-09T14:49:27+00:00 OK
2026-02-09T14:49:28+00:00 OK
```

14.1 Objetivo de este paso

Observar una reconvergencia STP real, con pérdida temporal de conectividad.

15 Medición de convergencia

El script `measure_convergence.py` analiza el log de pings (ejecutar en la segunda terminal dentro del contenedor):

```
python3 measure_convergence.py ping.log
```

Resultado esperado:

- Tiempo total de convergencia en segundos.

15.1 Objetivo de este paso

Cuantificar el impacto temporal de STP ante una falla.

16 Análisis de aprendizaje de direcciones MAC

Luego de la reconvergencia:

```
sh ovs-appctl fdb/show s4
```

Analizar:

- Qué MAC fueron aprendidas.
- En qué puertos.
- Qué switches no aprendieron nada.

16.1 Objetivo de este paso

Demostrar que el aprendizaje de direcciones MAC depende del árbol STP activo.

17 Captura de tráfico con tcpdump/wireshark

Para observar las tramas de la red en tiempo real sin interferir con la CLI de Container-net, las capturas deben ejecutarse directamente desde una nueva terminal en el host (fuera del contenedor) usando `docker exec -privileged`.

El flag `-l` (line-buffered) es fundamental para asegurar que `tcpdump` envíe y muestre cada trama inmediatamente en la salida estándar sin esperar a que se llene el buffer del sistema operativo.

17.1 Opción con tcpdump en consola

Desde una nueva terminal en su máquina host, ejecute la captura directa sobre el contenedor:

```
docker exec --privileged taller-stp tcpdump -l -e -nn -i s4-eth4 ether dst 01:80:c2:00:00:00
```

O para capturar todo el tráfico en dicha interfaz (incluyendo ARP e ICMP):

```
docker exec --privileged taller-stp tcpdump -l -e -nn -i s4-eth4
```

Mientras se está capturando, inyecte una falla en el enlace (ver sección 14) para observar el cese y reanudación del tráfico. Puede detener la captura en cualquier momento con `Ctrl+C`.

17.2 Opción con Wireshark

Alternativamente a la consola, se puede usar Wireshark para observar las tramas y BPDUs en tiempo real con interfaz gráfica. Para eso, ejecute en una terminal nueva en el host:

```
docker exec --privileged taller-stp tcpdump -nl -s 2048 -w - -i s4-eth4 | wireshark -k -i -
```

Mientras se está capturando, inyecte nuevamente una falla (ver sección 14).

17.3 Opción para guardar captura a archivo .pcap

Las tramas capturadas se pueden guardar en un archivo para su inspección posterior. En caso de usar Wireshark, se puede lograr esto deteniendo la captura (Capture, Stop en el menú) y después eligiendo dónde guardar el fichero .pcap (File, Save As).

En caso de estar usando `tcpdump`, se puede guardar la captura ejecutando:

```
docker exec --privileged taller-stp tcpdump -l -U -s 0 -w /workshop/results/stp_traffic.pcap -i s4-eth4
```

Para detener la captura presione `Ctrl+C`. Luego puede inspeccionar el archivo con:

```
docker exec --privileged taller-stp tcpdump -r /workshop/results/stp_traffic.pcap -e -nn
```

O alternativamente, copiando el fichero .pcap al host y abriéndolo con Wireshark.

17.4 Objetivo de este paso

La captura permite observar:

- BPDUs STP.
- ARP.
- Tráfico ICMP.

Relacionar teoría STP versus tráfico real de red.

18 Conclusiones

Al finalizar el taller, se debe comprender que:

- STP es un algoritmo distribuido, no un simple bloqueador de puertos.
- La topología física no siempre coincide con la topología lógica.
- No toda falla produce reconvergencia observable.
- Los timers son críticos para la estabilidad.
- STP condiciona forwarding, aprendizaje y latencia.
- Medir convergencia requiere entender el árbol, no solo bajar enlaces.

19 Referencias bibliográficas

- Larry Peterson and Bruce Davie, *Computer Networks: A Systems Approach*, Elsevier, 2022. <https://github.com/SystemsApproach/book>. Licencia: CC BY 4.0.
- Maarten van Steen and Andrew S. Tanenbaum, *Distributed Systems: Principles and Paradigms*, 4th Edition, Version 01, 2023. <https://www.distributed-systems.net/index.php/books/ds4/>.
- M. Peuster, H. Karl, and S. v. Rossem, *MeDICINE: Rapid Prototyping of Production-Ready Network Services in Multi-PoP Environments*. IEEE NFV-SDN, Palo Alto, USA, pp. 148–153, 2016. DOI: <https://doi.org/10.1109/NFV-SDN.2016.7919490>.